

เนื้อหาสไลด์นำเสนอ: ระบบจริงตามเกณฑ์ Data Engineering ข้อ 2-6

แหล่งข้อมูล: docs/evaluation/system-actual-analysis-topics-2-6.md เท่านั้น (ไม่ได้วิเคราะห์โค้ดใหม่) ไม่มีการให้คะแนน · อ้างอิงเป็น ไฟล์:บรรทัด ตามเอกสารต้นทาง

สัญลักษณ์สถานะ (ใช้ทุกสไลด์)

● REAL ทำงานจริง · ● PARTIAL ทำงานบางส่วน · ● UI ONLY มี UI แต่ logic จริงไม่ครบ · ● MOCK ข้อมูลจำลอง · ● NOT FOUND ไม่มีในระบบ

กฎการนำเสนอ: ห้ามพูดถึง ● ● ● เหมือนเป็น feature ที่ทำงานจริง ถ้าจะพูดให้พูดว่า "ยังไม่มี/เป็นข้อจำกัด" (รายการที่ห้ามอ้างรวมไว้ท้ายเอกสาร)

Slide 1 — ภาพรวมระบบ

จุดประสงค์: ให้คนฟังรู้ว่า SDOQAP คืออะไร และข้อมูลไหลผ่านระบบอย่างไรตั้งแต่ต้นจนจบ

เนื้อหาบน Slide:

- **SDOQAP** = Scalable Data Observability and Quality Assurance Platform
- ระบบ Data Engineering แบบ end-to-end: รับข้อมูล → ตรวจ/แปลง → เก็บ → นำไปใช้
- Stack จริง: React UI · FastAPI · Spark + Delta Lake · HDFS · Elasticsearch · n8n · LLM (Groq/Ollama)
- หลักคิด: ข้อมูลที่ไม่ผ่านเกณฑ์ **ไม่ถูกทิ้ง** แต่แยกเข้า Quarantine พร้อมเหตุผล
- ● ทุกชั้นใน Flow ด้านล่างมีโค้ดจริง · ● Reddit stream เป็นเส้นแยกที่ยังไม่เข้า Quality Engine

Diagram / Flow ที่ควรใส่:

Data Source → Extraction → Transformation → Data Quality → Loading → Utilization
(CSV·API·DB) (API+HDFS) (Spark stages) (แยก Valid/Quarantine) (Delta+ES) (Dashboard·Export)

- ใต้กล่อง Data Quality ใส่หมายเหตุเล็ก: "ตรวจรายแถวก่อนโหลด · คิดคะแนนหลังโหลด" (ดู Slide 5)
- ลูกศรแยกเส้นประ: Reddit → Kafka → Spark Streaming → Parquet + ES ป้าย ● "ไม่เข้า Quality Engine"

ข้อมูลจากระบบจริงที่ใช้สนับสนุน:

- Pipeline หลัก: SQE:run_quality_check (spark_quality_engine.py:1551-1811), ลำดับ stage S/pipeline/plan.py:2-6
- จุดเข้า: API/pipeline.py (ingest_csv/api/rdbms , land_and_queue 271-303)
- ปลายทาง: Delta /data/active , /data/quarantine ; ES sdoqap_*

บทพูด (~40 วินาที): "ระบบของเราชื่อ SDOQAP เป็นแพลตฟอร์มที่รับข้อมูลจากหลายแหล่ง ตรวจสอบคุณภาพและแปลงข้อมูลด้วย Spark แล้วเก็บเป็น Delta Lake บน HDFS สิ่งที่ต่างจาก ETL ทั่วไปคือข้อมูลที่ไม่ผ่านเกณฑ์จะถูกแยกเข้า Quarantine พร้อมเหตุผล ไม่ได้ถูกลบทิ้ง จากนั้นข้อมูลที่ผ่านแล้วถูกนำไปสร้าง Dashboard ได้โดยตรง วันนี้จะพาดูตามเกณฑ์ข้อ 2 ถึง 6 โดยทุกอย่างที่พูดมาจากโค้ดที่ทำงานจริง"

คำถามที่อาจารย์อาจถาม:

1. ระบบรองรับข้อมูลอะไรบ้าง? → ไฟล์ CSV/Excel, REST API, PostgreSQL ที่ต่อครบจริง; Reddit stream มีแต่เป็นเส้นแยก
2. ทุกอย่างใน Flow ทำงานจริงหมดไหม? → ใช่ในเส้นหลัก; ส่วนที่เป็น mock หรือยังไม่มี เราระบุไว้ในสไลด์ที่เกี่ยวข้อง (Sell-In/Out เป็น mock, ไม่มี Grafana dashboard)

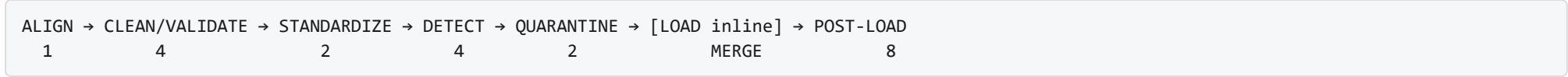
Slide 2 — จำนวนกระบวนการจัดการข้อมูล (เกณฑ์ข้อ 2 · 15 คะแนน)

จุดประสงค์: แสดงความครอบคลุมของกระบวนการจัดการข้อมูล โดยจัดกลุ่มให้เข้าใจง่าย ไม่ใช่โชว์แค่เลข 21

เนื้อหาบน Slide:

- **Pipeline หลักมี 21 stages** ลงทะเบียนจริงใน registry: ALIGN 1 · TRANSFORM 12 · POST_LOAD 8 ●
- จัดเป็น 6 กลุ่มการทำงาน: 1. **ALIGN** — จัดชื่อ/ชนิดคอลัมน์ให้ตรง schema (schema_align) 2. **CLEAN & VALIDATE** — schema_drift · auto_clean · validation · dedup 3. **STANDARDIZE** — standardize_dates ● · standardize_categories ● 4. **DETECT** — range_rules · anomaly_iqr · anomaly_zscore · anomaly_induced ● 5. **QUARANTINE & FILTER** — quarantine_assembly · column_filter 6. **POST-LOAD** — distribution · quarantine_breakdown · copdq · freshness · quality_score · ai_advisory · operational_impact · report
- **นอกจาก 21 stage** ยังมีขั้นตอน inline 7 ขั้น: ขอ lock · อ่าน raw · เขียน Quarantine · MERGE Active · OPTIMIZE/VACUUM · trigger Gold · archive raw
- และกระบวนการฝั่ง API/daemon อีกอย่างน้อย 10 รายการ เช่น ตรวจสอบ checksum ซ้ำ, อนุมาน schema+PK ตารางใหม่, คิวงาน+lock, auto-remediation, semantic auto-learn, rule governance, profile drift (PSI), Gold layer
- ทุก stage **ทำงานอัตโนมัติ** หลังผู้ใช้ส่งข้อมูลเข้า

Diagram / Flow ที่ควรใส่:



แถบสีตามกลุ่ม ตัวเลขใต้กล่องคือจำนวน stage; กล่อง LOAD วาดเป็นเส้นประเพื่อบอกว่าเป็น inline ไม่ใช่ stage ที่ลงทะเบียน

ข้อมูลจากระบบจริงที่ใช้สนับสนุน:

- S/pipeline/plan.py:2-6 (รายชื่อ 3 ช่วง), registry.py:17-32 (ลงทะเบียน + จับเวลา stage_seconds)
- SQE:1551-1811 (ลำดับ inline), trigger_core.py:53-82 (คิว FIFO ต่อตาราง)
- รายละเอียดแต่ละ stage: S/stages/schema.py, cleansing.py, standardize.py, rules.py, anomaly.py, assembly.py, metrics.py, advisory.py, report.py

บทพูด (~50 วินาที): "Pipeline หลักของเรามี 21 stage แบ่งเป็น 3 ช่วงคือ ALIGN 1, TRANSFORM 12 และ POST-LOAD 8 แต่เพื่อให้เห็นภาพ ผมจัดเป็น 6 กลุ่ม ตั้งแต่จัดโครงสร้างข้อมูล ตรวจสอบและล้างข้อมูล มาตรฐานค่า ตรวจสอบจับค่าผิดปกติ แยกข้อมูลที่ไม่ผ่านเข้า Quarantine และคำนวณเมตริกหลังโหลด นอกจาก 21 stage ยังมีขั้นตอนที่ฝังในโค้ดอีก เช่น การสื่อกตาราง การ MERGE และการ archive ไฟล์ดิบ รวมถึงกระบวนการฝัง API เช่น ตรวจสอบไฟล์ซ้ำด้วย SHA-256 ทั้งหมดรันอัตโนมัติ"

คำถามที่อาจารย์อาจถาม:

1. 21 stage ทำงานจริงทุกตัวไหม? → ลงทะเบียนและถูกเรียกครบ แต่บางตัวผลน้อย: standardize_categories ต้องมีกฎใน registry ซึ่งใน repo ไม่มีตัวเขียนค่านี้, anomaly_induced มักว่างเพราะต้องมีกฎที่คนอนุมัติ
2. ใครเป็นคนกำหนด logic? → ส่วนใหญ่ Dev ฮาร์ดโค้ด; กฎ range/IQR/DSL เป็น User ตั้งค่า; LLM เสนอกฎได้แต่ต้องมีคนอนุมัติ (ยกเว้น auto-remediation ที่ใช้อัตโนมัติเมื่อ confidence ≥ 0.80)
3. นับ "จำนวนกระบวนการ" อย่างไร? → ตอบตามขั้น: 21 stage ใน pipeline; + 7 ขั้น inline; + กระบวนการฝัง API/daemon ≥ 10 — ขอให้ระบุว่ามีขั้นไหน

Slide 3 — Data Extraction (เกนท์ข้อ 3 · 10 คะแนน)

จุดประสงค์: ชี้นำดึงข้อมูลจากแหล่งไหนได้จริง ตรวจสอบอะไรก่อนรับเข้า และแยกให้ชัดว่าเส้นทางไหนยังไม่เข้า Quality Engine

เนื้อหาบน Slide:

- แหล่งที่รองรับจริง
 - ● CSV (อัปโหลด) · ● Excel .xlsx/.xls (แปลงเป็น CSV, อ่าน sheet แรก)

- ● REST API (JSON/CSV, มี resolver data.go.th) · เรียกผ่าน API ตรง/n8n
- ● PostgreSQL (SELECT เท่านั้น, ต้องอยู่ใน allowlist)
- **เส้นทางแยก:** ● Reddit → Kafka → Spark Streaming → Parquet + ES — ข้อมูลลงจริงแต่ **ไม่เชื่อม Quality Engine**
- **สิ่งที่ตรวจจริงก่อนรับเข้า (ที่ API)**
 - ● Authentication (session / service key)
 - ● ชื่อตาราง (regex กัน path traversal)
 - ● ขนาดไฟล์ ≤ 200 MB (เกิน → HTTP 413)
 - ● ไฟล์ว่าง (→ HTTP 400)
 - ● SHA-256 + ตรวจ ingest ซ้ำ
 - ● คอลัมน์ PK ในหัวไฟล์ (เมื่อมี PK ลงทะเบียนแล้ว)
 - ● SSRF guard (REST) / SELECT-only + read-only + timeout 30s + จำกัด 1,000,000 แถว (DB)
- **ตรวจหลังลงไฟล์ที่ Spark:** schema, ชนิดข้อมูล, null, ซ้ำ ระดับแถว (ดู Slide 4)
- **ข้อจำกัดที่รู้:** ● ไม่เทียบจำนวน record ต้นทาง vs ที่รับ · ● ไม่ตรวจ encoding/delimiter · ● ไม่รองรับ JSON/Parquet/Avro/XML (ไฟล์) และ DB อื่นนอกจาก PostgreSQL
 - ● REST ผัง API ไม่มี pagination/retry
- ● แท็บ "API" และ "Stream" บนหน้า Ingestion เป็นโหมดสาธิต (จำลอง) — **ไม่นับเป็นการ ingest จริง**

Diagram / Flow ที่ควรใส่:

```

Input (CSV/Excel · REST · PostgreSQL)
→ Guard / Validation (auth · ชื่อตาราง · ขนาด · ว่าง · SHA-256 · PK header · SSRF/SQL guard)
→ Normalize (Excel/JSON/DB → CSV)
→ HDFS Raw /data/raw/<table>/<ingest_id>/
→ Run Registry (ES sdoqap_runs: QUEUED)
→ Spark daemon → spark-submit
  
```

เส้นแยกด้านล่าง: Reddit → Kafka → Spark Streaming → Parquet + ES (ป้าย ● ไม่เข้า Quality Engine)

ข้อมูลจากระบบจริงที่ใช้สนับสนุน:

- API/pipeline.py:315-333 (ingest_csv), :335-508 (ingest_api), :567-613 (ingest_rdbms), :194-227 (upload_to_webhdfs, retry 5 ครั้ง), :271-303 (land_and_queue)
- API/ingest_guards.py:22-109 (SSRF, SQL guard, ขนาดไฟล์), validation.py:4-18 (ชื่อตาราง)
- run_registry.py:21-65 (SHA-256, สถานะ run), สถานะ QUEUED → RUNNING → SUCCEEDED/FAILED/SKIPPED/TRIGGER_FAILED

- `streaming_job.py`, `reddit_stream.py` (stream แยก); `whitebox.py:1593-1618` (ingest-source จำลอง)

บทพูด (~55 วินาที): "ระบบรับข้อมูลได้จริง 3 แบบ คือไฟล์ CSV หรือ Excel, REST API และ PostgreSQL ก่อนรับเข้า API จะตรวจสอบสิทธิ์ ชื่อตาราง ขนาดไม่เกิน 200 เมกะไบต์ ไฟล์ไม่ว่าง และคำนวณ SHA-256 เพื่อกันการนำเข้าไฟล์ซ้ำ ถ้าตารางมีคีย์หลักลงทะเบียนไว้ก็ตรวจสอบว่าไฟล์มีคอส้มนั้นด้วย จากนั้นแปลงทุกแบบเป็น CSV เก็บไว้ใน HDFS เป็นโพลเดอร์ต่อการนำเข้า บันทึกสถานะลง Elasticsearch แล้วส่ง Spark ทำงานต่อ ส่วน Reddit stream มีจริงแต่เป็นอีกเส้นทางหนึ่ง ยังไม่ผ่าน Quality Engine และต้องบอกตรง ๆ ว่ายังไม่มีการเทียบจำนวนแถวต้นทางกับที่รับเข้า"

คำถามที่อาจารย์อาจถาม:

1. ถ้า *extraction* ผิดพลาดเกิดอะไรขึ้น? → ตรวจไม่ผ่านที่ API ตอบ 400/413 ไม่ลงไฟล์; เขียน HDFS ล้มเหลว retry 5 ครั้งแล้ว 500; daemon ติดต่อไม่ได้ → 503 และ run เป็น `TRIGGER_FAILED` ไปด้วย `POST /pipeline/retry/<ingest_id>`; Spark ล้ม → `FAILED raw` ยังอยู่ รันซ้ำได้
2. รู้ได้อย่างไรว่าดึงมาครบ? → ตอบตรง ๆ: ยังไม่มีการเทียบจำนวน record ต้นทางกับที่รับ (บันทึกเพียง `size_bytes`; `rows_ingested` มีเฉพาะ response ของ PostgreSQL) และไม่ได้อ่านกลับตรวจไฟล์ใน HDFS
3. ข้อมูลซ้ำกันแล้วทำอย่างไร? → ไฟล์ซ้ำ (checksum เดียวกัน, สถานะ `QUEUED/RUNNING/SUCCEEDED`) ตอบ `status: duplicate` ไม่ลงซ้ำ; มีข้อจำกัดคืออัปโหลดซ้ำพร้อมกันอาจผ่านทั้งคู่ และเส้นทาง n8n ไม่ผ่านการตรวจนี้

Slide 4 — Data Transformation (เกณฑ์ข้อ 4 · 10 คะแนน)

จุดประสงค์: แสดง transformation ที่ทำจริงด้วยตัวอย่าง Before → Logic → After ไม่ให้แน่น เลือก 3 ตัวอย่างหลัก

เนื้อหาบน Slide:

- **Transformation ที่ระบบทำจริง** (🟢 ถ้าไม่ระบุ) 1. แปลงชนิดข้อมูล (Integer/Double/Timestamp 6 รูปแบบ + epoch) 2. Rename คอลัมน์แบบ fuzzy ให้ตรงชื่อใน registry 3. ลบซ้ำตาม PK (2 ชั้น: `auto_clean` + `dedup`) 4. Validation: PK ว่าง · NULL · ชนิดไม่ถูก · วันที่ว่าง 5. มาตรฐานวันที่ รวม พ.ศ. → ค.ศ. 6. Range / IQR / Z-score (3.0) → Quarantine 7. แยก Valid / Invalid (ระดับแถว)
- 🟡 เติมค่าว่าง: มีใน DSL (`fillna` / `calculate`) แต่ไม่มี mean/median/mode · 🟡 Join/Aggregation บนข้อมูลธุรกิจ: ไม่มีใน Spark path (Dashboard Builder ทำ aggregation ด้วย pandas ตอนแสดงผล)
- **3 ตัวอย่าง Before → Logic → After**

#	Before	Logic	After
ก	"1,234" (string)	Integer = ตัดอักขระที่ไม่ใช่ตัวเลข → cast (schema.py:64-68)	1234 (Integer)
ข	22-7-2569 · 20/07/2026 · 14 Jul 2026	ปี > 2500 ลบ 543, แปลงทุกรูปแบบ (standardize.py:6-57)	2026-07-22 · 2026-07-20 · 2026-07-14
ค	users.csv 5 แถว: id=2 ซ้ำ, แถวไม่มี id, id=3 email ว่าง	auto_clean ลบซ้ำ → validation	ซ้ำถูกลบ · missing_primary_key · null_value_in_email → Quarantine

- ข้อจำกัดที่รู้ (พูดเองก่อนถูกถาม): แถวที่ auto_clean ลบซ้ำ ไม่ถูกนับใน quarantine หรือ total · วันที่ไม่ตรวจสอบความถูกต้องของปฏิทิน และใช้กับคอลัมน์ชื่อ วันที่ / date / Date เท่านั้น

Diagram / Flow ที่ควรใส่:

```
Raw (string ทั้งหมด) → schema_align → schema_drift → auto_clean → validation → dedup
→ standardize_dates → range / IQR / Z-score → T Valid → clean_df
                                     L Invalid → Quarantine (พร้อม reject_reason)
```

ข้อมูลจากระบบจริงที่ใช้สนับสนุน:

- S/stages/schema.py:10-98 (cast/rename), cleansing.py:6-44, 47-115, 118-138, standardize.py:6-57, rules.py:15-33, anomaly.py:6-75, assembly.py:6-49
- ตัวอย่างจาก unit test: test_stage_schema.py:6-21, test_stage_cleansing.py:8-12, 31-37, 40-43; ไฟล์ตัวอย่าง services/spark/users.csv, grocery_sales.csv (ผลที่อ่านจาก logic/test ยังไม่ได้รับซ้ำ)
- ผู้กำหนด rule: Dev (ส่วนใหญ่) · User (range_checks, value_range, DSL) · System (อนุมาน schema/PK)

บทพูด (~55 วินาที): "หลังข้อมูลลงมา Spark อ่านทุกคอลัมน์เป็นข้อความก่อน แล้วค่อยแปลงชนิดตาม schema ตัวอย่างเช่น ค่า 1,234 ที่เป็นข้อความถูกแปลงเป็นเลข 1234 วันที่หลายรูปแบบรวมถึงปี พ.ศ. ถูกแปลงเป็นรูปแบบเดียวคือปี ค.ศ. จากนั้นระบบลบแถวซ้ำตามคีย์หลัก ตรวจสอบ null ตรวจสอบชนิด และตรวจค่าผิดปกติด้วยช่วงค่า IQR และ Z-score แถวที่ไม่ผ่านจะถูกแยกเข้า Quarantine พร้อมเหตุผล เช่นไฟล์ users.csv ที่มีแถวซ้ำ แถวไม่มี id และแถวที่ email ว่าง ต้องบอกด้วยว่าแถวที่ถูกลบเพราะซ้ำในขั้นแรกยังไม่ถูกนับรวมใน Quarantine"

คำถามที่อาจารย์อาจถาม:

1. *รู้ได้อย่างไรว่า transform ถูกต้อง?* → มี unit test ราย stage (`tests/unit/test_stage_*.py`) และ integration test ตรวจสอบ total/clean/quarantine/score; ที่ runtime ยังไม่มี reconciliation แยกเข้า = แยกออก + ถูกปฏิเสธ (`total_records = clean + quarantine`)
2. *จัดการค่าว่างอย่างไร?* → ค่าเริ่มต้น: NULL ในคอลัมน์ schema → Quarantine (`null_value_in_<col>`); ทางเติมค่าต้องตั้งกฎ DSL เอง (`fillna / calculate`); กฎตัวอย่างของ `student_course_scores` มี `fillna 0` ก่อน `calculate ... IS NULL` ทำให้เงื่อนไขหลังไม่ทำงาน (แถวคะแนนว่างกลายเป็น 0) — เป็นข้อสรุปจากการอ่าน logic
3. *แก้ปัญหาอะไรไม่ได้บ้าง?* → ไม่มี regex เป็น rule ผู้ใช้; `column_overrides` ของ IQR ไม่ถูกอ่าน; regex ของ Integer อาจทำให้ `"12.00"` เป็น `1200` (จากการอ่านโค้ด ยังไม่ได้รัน)

Slide 5 — Data Quality & Quality Gate

จุดประสงค์: อธิบายกฎคุณภาพที่ใช้จริง สูตรคะแนน และเปิดเผยข้อค้นพบสำคัญว่า quality score ไม่ได้ขวางการไหล

เนื้อหาบน Slide:

- **กฎคุณภาพ (ทุกข้อ → Quarantine พร้อม reject_reason)** ●
 - Missing PK → `missing_primary_key`
 - NULL → `null_value_in_<col>`
 - Invalid Type → `invalid_type_<col>`
 - Duplicate PK → `duplicate_records`
 - Out of Range → `out_of_range_<col>` (ต้องตั้ง `range_checks`)
 - IQR / Z-score (> 3.0) outlier → Quarantine
- **สูตรคะแนน:** $Quality\ Score = Clean \div (Clean + Quarantine) \times 100$ ●
- **Threshold:** ค่าเริ่มต้น 90, พื้น 70; โหมด adaptive = $\max(\text{mean}(15\ \text{รอบล่าสุด}) - \text{stdev}, \text{พื้น})$
- **ข้อค้นพบสำคัญ:** `quality_score` เป็น stage **POST_LOAD** — ถูกคำนวณ หลัง เขียน Quarantine และ MERGE เข้า Active แล้ว
 - ระดับแถว: ● มี gate ก่อนไหลจริง (แถวไม่ผ่านถูกแยกก่อนเขียน)
 - ระดับตาราง: ● **ไม่มี gate ที่หยุดการไหล**
- **คะแนนทำอะไรได้จริง:** ส่ง alert · ติดป้ายสถานะ `warnings` · ตัดสินว่าจะ rebuild Gold หรือไม่ · เรียก AI advisor (เมื่อ $\text{score} < \text{threshold} - 15$)
- **Gate ฝั่งผู้บริโภค:** ● `GET /lineage/{table}/trust-check` → SAFE / WARNING_SUSPECT / HALT_INGEST (advisory ผู้เรียกเลือกใช้เอง)
- มิติที่วัดจริง: Completeness · Uniqueness · Validity ● · Timeliness ● (วัดแต่ไม่เทียบเกณฑ์) · Consistency/Accuracy ● (ในเส้นหลัก)
- ● สวิตช์ `null_primary_key / duplicate_check / null_date_column` และ `null_checks` ใน rules ไม่ถูกอ่านโดย engine

Diagram / Flow ที่ควรใส่:

```
Transform → แยก Valid / Invalid (gate ระดับแถว) → เขียน Quarantine → MERGE Active
↓ (หลังโหลด)
quality_score → alert · state · Gold
```

ใส่กรอบแดงชี้ที่ "quality_score" พร้อมข้อความ "ไม่หยุดการไหล"

ข้อมูลจากระบบจริงที่ใช้สนับสนุน:

- S/stages/cleansing.py:47-135, rules.py:6-33, anomaly.py:6-118
- สูตรคะแนน metrics.py:168-175; alert :177-183; Z-score อัตรา quarantine :184-210 (ต้องมี ≥ 3 รอบ)
- threshold dynamic_rules_engine.py:249-335; ลำดับ POST_LOAD plan.py:5-6; ลำดับเขียน SQE:1704-1735 แล้วคิดคะแนน :1774
- Gold trigger SQE:1777-1788; state report.py:75; trust-check API/lineage.py:340-447

บทพูด (~55 วินาที): "กฎคุณภาพของเราตรวจรายแถว ถ้าคีย์หลักว่าง ค่าว่าง ชนิดผิด ข้ำ เกินช่วง หรือเป็นค่าผิดปกติ แแถวนั้นจะเข้า Quarantine พร้อมเหตุผล คะแนนคุณภาพคือสัดส่วนแถวที่ผ่านต่อแถวทั้งหมด จุดที่ต้องบอกอย่างตรงไปตรงมาคือคะแนนนี้คำนวณหลังโหลด ไม่ได้เป็นประตูที่หยุดการไหล ตัวที่ทำหน้าที่ประตูลงคือการแยกแถวผ่านหรือไม่ผ่านก่อนเขียน ส่วนคะแนนใช้ส่งการแจ้งเตือน ติดสถานะ และตัดสินใจว่าจะสร้างตารางสรุป Gold หรือไม่ ผู้ใช้ปลายทางสามารถเรียก trust-check เพื่อดูว่าตารางนี้ปลอดภัยพอจะใช้หรือไม่"

คำถามที่อาจารย์อาจถาม:

1. ทำไมไม่หยุดการไหลเมื่อคะแนนต่ำ? → การออกแบบปัจจุบันแยกแถวไม่ผ่านออกก่อนแล้วไหลเฉพาะแถวที่ผ่าน; คะแนนทำหน้าที่เตือนและควบคุมขั้นต่อไป (Gold) ไม่ใช่หยุดไหล — ตารางที่คะแนนต่ำยังถูก MERGE แถว valid เท่าที่มี
2. threshold ตั้งอย่างไร? → ค่าเริ่มต้น 90 พื้น 70; แบบ adaptive ใช้ประวัติของตารางนั่นเอง (จึงทำให้ตารางที่เคยแยกลดลงได้ แต่ไม่ต่ำกว่าพื้น)
3. มติคุณภาพครบไหม? → มี Completeness/Uniqueness/Validity เป็นกฎบังคับ; Timeliness วัดแล้วแต่ไม่เทียบเกณฑ์; Consistency และ Accuracy ยังไม่มีในเส้นหลัก (Accuracy มีเฉพาะ benchmark ของ whitebox กับข้อมูลคะแนนนักเรียน)

Slide 6 — Data Loading (เกณฑ์ข้อ 5 · 5 คะแนน)

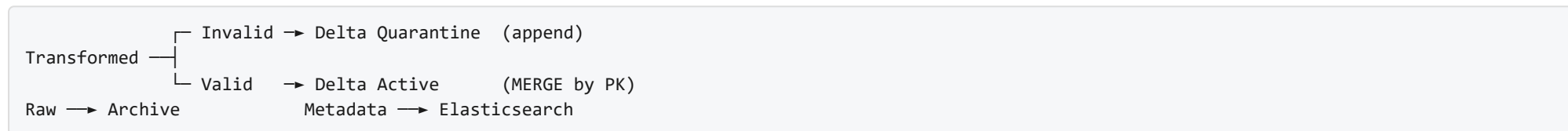
จุดประสงค์: แสดงว่าข้อมูลแต่ละประเภทไปที่ไหน วิธีเขียน และข้อจำกัดที่พบจริง

เนื้อหาบน Slide:

- ปลายทางจริง

- ● Valid → **Delta Active** /data/active/<table> (MERGE ตาม PK; รอบแรก overwrite)
- ● Invalid → **Delta Quarantine** /data/quarantine/<table> (append, partition ตาม run_id, ลบแถวของ ingest_id เดิมก่อน)
- ● Raw → **Archive** /data/archive/<table>/<ingest_id> หลังรันสำเร็จ
- ● Metadata → **Elasticsearch**: sdoqap_quality_runs, lineage_runs, pipeline_runs, runs, locks, registries
- ● Gold: sdoqap_gold_* เป็นสรุปจาก metadata ของ run ไม่ใช่ข้อมูลธุรกิจ
- **กลไก**: เขียน Quarantine ก่อน แล้วค่อย MERGE Active · MERGE ล้ม → fail-closed ไม่แตะ Active · autoMerge ให้ schema ขยายได้ · OPTIMIZE/VACUUM ทุก 10 เวอร์ชัน (● อาจไม่ทำงานถ้าตารางไม่มีคอลัมน์ row_hash)
- **กันซ้ำ**: MERGE ตาม PK ● · ไฟล์ซ้ำด้วย SHA-256 ● · Quarantine ซ้ำกันเฉพาะเส้นทางที่มี ingest_id ●
- **ข้อจำกัดที่พบจริง**
 - ● ไม่มี rollback / Delta restore
 - ● ไม่เทียบจำนวนต้นทาง vs ปลายทาง (ไม่มี source-vs-destination reconciliation)
 - ● ไม่มี incremental จากต้นทาง (watermark/CDC) — มีแต่ upsert ฝั่งปลายทาง
 - ● ไม่มี Elasticsearch bulk และ ES ไม่ retry (เขียนทีละเอกสาร; ล้มเหลวแค่ print)
 - ● ไม่มี staging แล้ว rename เข้า Active
 - ● Postgres/Grafana ไม่ใช่ปลายทาง

Diagram / Flow ที่ควรใส่:



ติดเลข ① Quarantine ก่อน ② MERGE ทีหลัง; ลูกศรทั้งหมดอยู่บน HDFS ยกเว้น Metadata

ข้อมูลจากระบบจริงที่ใช้สนับสนุน:

- SQE:1704-1711 (Quarantine), :1713-1735 (MERGE/overwrite), :1737-1753 (fail-closed), :1756-1765 (OPTIMIZE/VACUUM), :1790-1805 (archive), :85 (autoMerge)
- S/stages/assembly.py:6-49, report.py:52-80, run_registry.py:50-68, S/common/keys.py:14-55 (PK อนุมัติ ≥ 90% ไม่ซ้ำ)
- ES เขียนทีละเอกสาร: SQE:96-121; spark_gold_layer.py:96-103

บทพูด (~50 วินาที): "ข้อมูลที่ผ่านมาถูกเขียนเข้า Delta Lake ตาราง Active ด้วยการ MERGE ตามคีย์หลัก ถ้ารันซ้ำก็อัปเดตแถวเดิมแทนที่จะซ้ำ ข้อมูลที่ไม่ผ่านถูกเขียนต่อท้ายใน Delta Quarantine เราเขียน Quarantine ก่อน แล้วค่อย MERGE ถ้า MERGE ล้มเหลวระบบจะไม่แตะตาราง Active ไฟล์ดิบถูกย้ายไป Archive และ Metadata ของการรันไปเก็บใน Elasticsearch ข้อจำกัดที่ต้องบอกคือ ยังไม่มี rollback ยังไม่มีการเทียบจำนวนแถวต้นทางกับปลายทาง และยังไม่ incremental load จากฝั่งต้นทาง"

คำถามที่อาจารย์อาจถาม:

1. *รู้ได้อย่างไรว่าไหลครบ?* → ตอบตรง ๆ ว่ายังไม่มีการเทียบจำนวนแถวที่ส่ง vs ที่ลงปลายทาง; `total_records` = clean + quarantine เป็นความสอดคล้องภายในไม่ใช่ความครบเทียบต้นทาง; สถานะ SUCCEEDED หมายถึง exit code 0
2. *รันซ้ำแล้วข้อมูลซ้ำไหม?* → Active ไม่ซ้ำเพราะ MERGE ตาม PK; Quarantine ไม่ซ้ำเมื่อมี `ingest_id`; เส้นทางเก่า (n8n) อาจ append ซ้ำ; metadata ใน ES เพิ่มเอกสารใหม่ทุกครั้ง จึงอาจทำให้ Gold นับซ้ำ
3. *ถ้าไหลล้มเหลวครึ่งทาง?* → MERGE ล้ม = fail-closed Active ไม่ถูกแตะ แต่ Quarantine ที่เขียนไปแล้วคง commit ไว้ และรันซ้ำ `ingest_id` เดิมจะลบแล้วเขียนใหม่; ไม่มี rollback/restore อัตโนมัติ

Slide 7 — Data Utilization (เกนท์ข้อ 6 · 5 คะแนน)

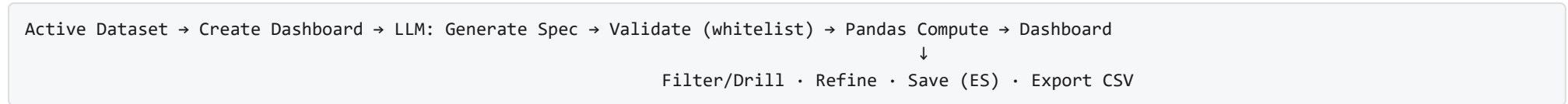
จุดประสงค์: แสดงการใช้ข้อมูลผ่าน pipeline จริง โดยแยก REAL / PARTIAL / MOCK ชัดเจน และอธิบายบทบาทของ LLM ให้ถูก

เนื้อหาบน Slide:

- **Create Dashboard (AI)** ● — เลือก dataset จาก Active (ข้อมูลที่ผ่านมาแล้ว) → ผู้ใช้เขียนความต้องการ → LLM ออกแบบ spec → ตรวจสอบ spec → pandas คำนวณ → แสดงผล → โต้ตอบ/refine → บันทึก
 - **LLM ไม่ได้คำนวณตัวเลขเอง:** ออกแบบ "dashboard specification" เท่านั้น; pandas คำนวณ KPI/กราฟ
 - ส่งให้ LLM เฉพาะ profile (ชื่อคอลัมน์ ชนิด distinct missing% min/max) — **ไม่ส่งแถวข้อมูลและค่าหมวดหมู่**
 - ไม่มี key / LLM ล้ม → ใช้ rule-based fallback (refine ต้องใช้ LLM)
 - ผู้ใช้ตรวจสอบได้: ตัวนับ `rows_after_filter` / `rows_total`, ตัวอย่าง 20 แถว, ตารางแถวจริง, ป้ายบอกว่าใช้ `groq` หรือ `rules`
- **ส่วนอื่นที่ใช้ข้อมูลจริง**
 - ● Export CSV (active / quarantine / raw / gold) · ● Trust-check · ● Schema/Rule governance (อนุมัติ/ปฏิเสธ)
 - ● หน้า Dashboard 4 แท็บ: KPI สุขภาพข้อมูล/SLA/freshness/COPDQ คำนวณจาก ES จริง แต่บางส่วน mock
 - ● Analytics: forecast (regression), clustering (จับคีย์เวิร์ด), impact (ค่าคงที่สมมติ)
- **ส่วนที่ไม่ใช่ของจริง (บอกอย่างตรงไปตรงมา)**
 - ● Sell-In/Sell-Out (timeline ฮาร์ดโค้ด), narrative ธุรกิจบางส่วน, metadata ของ lineage node

- ● ปุ่ม "นำไปใช้" ของ Recommendation (ไม่มี backend halt/notify/restore)
- ● Grafana dashboard (มีแค่ datasource + alert rules), Prometheus ที่รันจริง

Diagram / Flow ที่ควรใส่:



ได้ลูกศร LLM ใส่ป้าย "ไม่คำนวณตัวเลข" และให้ pandas ใส่ "คำนวณจริง"; ด้านล่างทำแถบ legend ●●●●● ระบุ feature แต่ละตัว

ข้อมูลจากระบบจริงที่ใช้สนับสนุน:

- API/dashboards.py:73-185, dashboard_llm.py (Groq openai/gpt-oss-120b, temp 0.2, timeout 20 วินาที, retry 1 ครั้ง), dashboard_spec.py:191-216 (whitelist: 16 widget/6 filter), dashboard_compute.py:211-222, dashboard_data.py:157-171
- UI: DashboardBuilder.jsx, components/builder/*; บันทึกใน ES sdoqap_dashboards
- KPI/Analytics: API/analytics.py (:12-377, :460-597, :673-766, Sell-In/Out mock :768-831), trust-check API/lineage.py:340-447, export API/data_export.py:552-685
- test: services/api/tests/test_dashboard_*.py และ test_dashboards_*.py 8 ไฟล์ (ยังไม่ได้รัน)

บทพูด (~55 วินาที): "ปลายทางของข้อมูลในระบบคือการนำไปใช้ จุดเด่นคือ Create Dashboard ผู้ใช้เลือกชุดข้อมูลผ่านการตรวจแล้ว บอกความต้องการ แล้ว LLM ช่วยออกแบบโครงของแดชบอร์ด ระบบตรวจโครงนั้นกับรายการที่อนุญาต แล้ว pandas เป็นผู้คำนวณตัวเลขจริง LLM ไม่เคยเห็นแถวข้อมูล เห็นแค่ชื่อคอลัมน์และสถิติสรุป และไม่ได้คำนวณเลขเอง ถ้า LLM ใช้ไม่ได้ก็มีโหมดกฏอัตโนมัติ นอกจากนี้ยังมี Export และ Trust-check ส่วนหน้า Dashboard หลักมีทั้งตัวเลขจริงจาก Elasticsearch และบางส่วนที่เป็นข้อมูลจำลอง เช่นกราฟ Sell-In Sell-Out ซึ่งเราระบุชัดว่าเป็นตัวอย่าง"

คำถามที่อาจารย์อาจถาม:

1. ตัวเลขใน Dashboard เชื่อมโยงได้ไหม? → ใน Builder ตัวเลขมาจาก pandas บนตาราง Active โดยตรง ตรวจเทียบกับตัวอย่าง 20 แถว ตารางจริง และ CSV จาก /export/active/{table}; ส่วนหน้า Dashboard หลักมี mock (Sell-In/Out) และ narrative บางส่วนฮาร์ดโค้ด
2. ข้อมูลวิ่งไป LLM ไหม? → Builder ส่งเฉพาะ profile ไม่ส่งแถว/ค่าหมวดหมู่ (คอลัมน์คล้าย identifier ไม่ส่ง min/max); ข้อควรระวัง: auto_remediation_engine.py ส่งตัวอย่าง quarantine 5 แถวให้ LLM โดยไม่ปิดบัง (ต่างจาก advisor ที่ปิดบัง identifier)
3. Grafana ใช้ทำอะไร? → มี datasource ต่อ Elasticsearch และ alert rules เท่านั้น ไม่มี dashboard ที่ provision ไว้ UI ไม่ลิงก์ไป Grafana

Slide 8 — White Box: ใครทำอะไร?

จุดประสงค์: ตอบคำถามว่าแต่ละส่วนของระบบ "ใครเป็นผู้กำหนด/ลงมือ" เพื่อแสดงว่าเข้าใจกลไกภายในจริง

เนื้อหาบน Slide (4 คอลัมน์):

Developer	User	System / Spark	LLM
เขียนโค้ด pipeline 21 stage	อัปโหลดไฟล์ / ส่ง ingest (UI/API)	รัน spark-submit ตามคิว FIFO ต่อตาราง + ขอ lock	ออกแบบ Dashboard Specification (Groq)
กฎ validation ฮาร์ดโค้ด (PK ว่าง, NULL, type, date, duplicate)	ตั้งกฎ <code>range_checks</code> , <code>value_range</code> , DSL remediation	อนุมาน schema + PK ตารางใหม่ (PK ไม่ซ้ำ $\geq 90\%$)	เสนอกฎคุณภาพ (AI advisor: Groq $\rightarrow$ Ollama $\rightarrow$ heuristic) $\rightarrow$ <code>PROPOSED</code> ต้องมีคนอนุมัติ
Z-score 3.0, สูตร quality score, ลำดับ stage	อนุมัติ/ปฏิเสธ schema drift และ AI proposal	ตรวจ drift, anomaly (IQR/Z-score), cast, dedup	สร้างกฎ remediation (auto-remediation) — ใช้อัตโนมัติเมื่อ confidence ≥ 0.80
Guard: SSRF, SELECT-only, ชื่อตาราง, ขนาด 200 MB	สร้าง/บันทึก/refine แดชบอร์ด	MERGE Delta, เขียน quarantine, คำนวณ metrics, เขียน ES	เรียบเรียงข้อความ AI context บนหน้า Dashboard
ลำดับเขียน (Quarantine $\rightarrow$ MERGE)	retry run, แก้กฎ, resolve ticket, export CSV	ตรวจ checksum ซ้ำ, archive raw, ปลอย lock	ไม่ทำ: คำนวณตัวเลข Dashboard, alert, forecast, quality score

- ทุกข้อใน 4 คอลัมน์ ● ยกเว้น: rule induced / `standardize_categories` ● (ใช้น้อย) และ proposal ของ LLM ที่อนุมัติแล้วอาจไม่ merge เข้า `rules_config.json` ● (รูปแบบ rule ไม่ตรงกับชั้นอนุมัติ)
- n8n** (นอก 4 คอลัมน์): ตั้งเวลา ingest แหล่งตายตัวทุก 30 นาที, relay webhook, ส่ง alert — ● ต้อง import workflow เอง

Diagram / Flow ที่ควรใส่: ตาราง 4 คอลัมน์สี่ตัวต่างกัน ได้ตารางมีแถบ "ลูกศรการอนุมัติ": LLM $\rightarrow$ (เสนอ) $\rightarrow$ User $\rightarrow$ (อนุมัติ) $\rightarrow$ Rule ใช้งาน; และลูกศรแดงเส้นประ LLM $\rightarrow$ auto-remediation $\rightarrow$ ใช้เลย (ไม่ต้องอนุมัติ)

ข้อมูลจากระบบจริงที่ใช้สนับสนุน:

- Dev: `S/stages/cleansing.py`, `metrics.py:168-175`, `anomaly.py:58`, `API/ingest_guards.py`
- User: `API/dynamic_rules.py:99-192`, `472-636`, `API/schema.py:63-183`, `API/dashboards.py`
- System: `trigger_core.py:53-82`, `SQE:126-218` (lock), `S/common/keys.py`, `SQE:1702-1753`
- LLM: `dashboard_llm.py`, `ai_rule_advisor.py` / `S/stages/advisory.py:6-167`, `auto_remediation_engine.py:319-339`, `whitebox.py:1774-1856`

บทพูด (~45 วินาที): "ในมุมมอง White Box เราแยกชัดว่าใครทำอะไร นักพัฒนาเขียนกฎตรวจสอบพื้นฐานและลำดับ pipeline ผู้ใช้ส่งข้อมูล ตั้งกฎบางตัว อนุมัติการเปลี่ยน schema หรือกฎที่ AI เสนอ และสร้างแดชบอร์ด ส่วน Spark ทำงานจริงทั้งหมด ทั้งแปลงข้อมูล ตรวจสอบความผิดปกติ MERGE และคำนวณเมตริก LLM มีบทบาทสองอย่างคือออกแบบโครงสร้างแดชบอร์ดและเสนอกฎ ซึ่งส่วนใหญ่ต้องมีคนอนุมัติ ยกเว้นระบบแก้ไขอัตโนมัติที่ใช้กฎที่ความมั่นใจตั้งแต่ 0.80 ขึ้นไป และ LLM ไม่เกี่ยวกับการคำนวณตัวเลขหรือคะแนนคุณภาพ"

คำถามที่อาจารย์อาจถาม:

1. *AI แก่กฎเองได้ไหม?* → advisor เสนอเท่านั้น (PROPOSED) ต้องมีคนอนุมัติผ่าน guardrail (พื้น 70, ลดได้ $\leq 10\%$); แต่ auto-remediation เขียนกฎลง registry เองเมื่อ confidence ≥ 0.80 แล้วรันซ้ำ — เป็นจุดที่ไม่ผ่านคนอนุมัติ
2. *ใครกำหนดเกณฑ์คุณภาพ?* → Dev กำหนดสูตรและกฎพื้นฐาน; ค่า threshold เริ่มต้น 90 พื้น 70 และปรับแบบ adaptive ตามประวัติ; User ตั้ง range/IQR ต่อตารางได้
3. *Spark ทำงานเมื่อไร?* → อัปเดตใหม่หลังผู้ใช้/n8n ส่งข้อมูล: API เรียก daemon /retry → คิวต่อตาราง → spark-submit

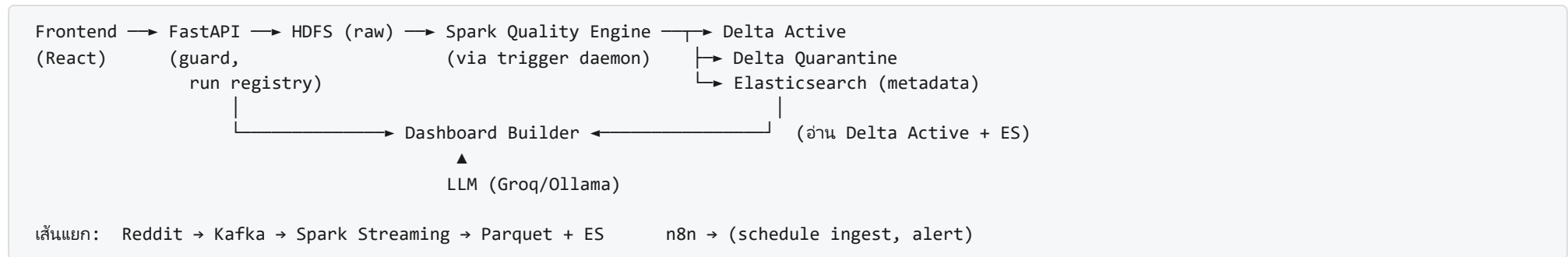
Slide 9 — Actual Architecture

จุดประสงค์: แสดงสถาปัตยกรรมที่ทำงานจริง โดยบอกหน้าที่ของแต่ละ component

เนื้อหาบน Slide:

- **Frontend (React UI)** ● — ingest, rules, pipeline, dashboard builder, export, schema governance
- **FastAPI** ● — auth, guard, landing ไฟล์ลง HDFS, run registry, ให้บริการ dashboard/analytics/export/lineage
- **Spark Trigger Daemon + Quality Engine** ● — คิวต่อตาราง, lock, รัน 21 stages, MERGE Delta
- **HDFS** ● — raw / archive / active / quarantine / Reddit Parquet
- **Delta Lake** ● — ตาราง Active (MERGE) และ Quarantine (append)
- **Elasticsearch** ● — metadata: runs, quality_runs, lineage, schema/rules registry, dashboards, Gold
- **LLM (Groq / Ollama)** ● — ออกแบบ dashboard spec, เสนอกฎ, สร้างกฎ remediation (Ollama ผูก compose profile ai และไม่พบสคริปต์ pull โมเดล → ยังไม่ยืนยันว่าพร้อมใช้ตอนรัน)
- **n8n** ● — ตั้งเวลา ingest แหล่งตายตัว + alert (ต้อง import workflow เอง)
- **Kafka + Spark Streaming** ● — เส้นแยกของ Reddit
- **Grafana** ● — มี datasource + alert rules; ● ไม่มี dashboard
- **Prometheus** ● — มีไฟล์ config แต่ไม่มี service ใน compose
- **PostgreSQL** — เป็น แหล่งข้อมูลเท่านั้น ไม่ใช้ปลายทาง

Diagram / Flow ที่ควรใส่:



ได้แต่ละกล่องใส่หน้าที่ 1 บรรทัด และใช้สี/ไอคอนสถานะตามข้างบน

ข้อมูลจากระบบจริงที่ใช้สนับสนุน:

- docker-compose.yml (487 บรรทัด; profile streaming, ai, tools), infra/n8n/ingestion_workflow.json, infra/grafana/provisioning/*, infra/prometheus/prometheus.yml
- API/pipeline.py:230-242 (API → daemon /retry), spark_trigger_daemon.py, trigger_core.py, SQE
- API/data_export.py:154-199 (อ่าน Delta ผ่าน WebHDFS+pandas), API/dashboard_data.py:157-171

บทพูด (~45 วินาที): "สถาปัตยกรรมจริงคือ ผู้ใช้เข้าทางหน้าเว็บ React ส่งข้อมูลให้ FastAPI ซึ่งตรวจแล้วเขียนไฟล์ดิบลง HDFS และสั่ง Spark ผ่านตัวกลางที่เรียกว่า trigger daemon Spark ประมวลผลแล้วเขียน Delta สองตาราง คือ Active กับ Quarantine และเขียน Metadata ลง Elasticsearch ฝั่งการใช้งาน Dashboard Builder อ่านตาราง Active และเรียก LLM ออกแบบโครง ส่วน n8n, Kafka และ Grafana มีอยู่ แต่บทบาทจำกัดตามที่ระบุ Grafana มีแค่การเชื่อมข้อมูลกับกฎแจ้งเตือน ยังไม่มีแดชบอร์ดสำเร็จรูป"

คำถามที่อาจารย์อาจถาม:

- ทำไมใช้ Delta Lake? → ได้ MERGE (upsert) ตาม PK, schema autoMerge, และตารางเดียวกันรองรับอ่านจาก API ผ่าน WebHDFS; เขียน Active ด้วย MERGE ล้มแล้วไม่แตะตารางเดิม (fail-closed)
- Elasticsearch เก็บอะไร? → metadata ไม่ใช่ข้อมูลระดับแถว: สถานะ run, quality metrics, lineage ระดับตาราง, registry ของ schema/rules, drift, dashboard ที่บันทึก, Gold aggregates; ไม่มี index template (dynamic mapping) ยกเว้น Gold 4 index
- ระบบรองรับหลายเครื่องหรือไม่ (scalability)? → มี Spark master/worker และไฟล์ infra/swarm/docker-swarm-ha.yml ในโปรเจกต์ แต่ผลวิเคราะห์ที่ใช้ทำสไลด์นี้ไม่ได้ตรวจสอบ HA/scale จึงไม่ยืนยัน — ตอบว่า "ยังไม่ได้ทดสอบ"

Slide 10 — สรุป End-to-End

จุดประสงค์: สรุปภาพเดียวให้คนฟังเห็นวงจรครบ และปิดด้วยประโยคจุดยืนของระบบ

เนื้อหาบน Slide (แต่ละขั้น 1 ประโยค):

- **SOURCE** — รับข้อมูลจาก CSV/Excel, REST API, PostgreSQL ● (Reddit stream เป็นเส้นแยก ●)
- ↓ **EXTRACTION** — API ตรวจสิทธิ์ ชื่อตาราง ขนาด ไฟล์ว่าง ไฟล์ซ้ำ แล้วลง HDFS Raw ต่อหนึ่ง ingest ●
- ↓ **TRANSFORMATION** — Spark แปลงชนิด จัดชื่อ ลบซ้ำ มาตรฐานวันที่ และตรวจค่าผิดปกติผ่าน 21 stages ●
- ↓ **QUALITY CHECK** — ฎรายแถวตัดสินผ่าน/ไม่ผ่าน; คะแนนคุณภาพคำนวณหลังโหลดเพื่อแจ้งเตือนและควบคุมขั้นต่อไป ● (ไม่ใช่ gate หยุดโหลด)
- ↓ **VALID / QUARANTINE** — แถวผ่านไป Active, แถวไม่ผ่านไป Quarantine พร้อม reject_reason ●
- ↓ **LOADING** — Delta MERGE ตาม PK + metadata ลง Elasticsearch + archive raw ● (ยังไม่มี rollback / reconciliation / CDC)
- ↓ **UTILIZATION** — สร้าง Dashboard ด้วย AI จากข้อมูลที่ผ่านแล้ว, Export, Trust-check ● (หน้า Dashboard หลักปนส่วน mock ●)
- **ปิดท้าย (ตัวหนา):**

"SDOQAP ไม่ได้ทำแค่ ETL แต่รวม Data Quality, Quarantine, Monitoring และการนำข้อมูลไปใช้ผ่าน Dashboard เข้าด้วยกัน"

- ● **หมายเหตุสถานะ Monitoring:** ตัวเก็บเมตริก/แจ้งเตือนมีในโค้ด แต่ปลายทางแจ้งเตือน (Slack/LINE) ต้องตั้งค่าเอง และ Grafana dashboard / Prometheus ยังไม่มี

Diagram / Flow ที่ควรใส่: แนวตั้ง 7 กล่องตามลำดับข้างบน ข้างแต่ละกล่องมีหมายเลขเกนซ์ (ข้อ 3, 4, 4-5, 5, 6) และสีสถานะ; ที่กล่อง QUALITY CHECK ใส่ไอคอนเตือนเล็ก "คะแนนหลังโหลด"

ข้อมูลจากระบบจริงที่ใช้สนับสนุน: สรุปจาก Slide 1-9; Workflow จริง docs/evaluation/system-actual-analysis-topics-2-6.md หัวข้อ 7

บทพูด (~45 วินาที): "สรุปภาพรวม ข้อมูลเข้ามาจากสามแหล่งหลัก ผ่านการตรวจที่ API ลงเก็บใน HDFS แล้ว Spark แปลงและตรวจผ่าน 21 stage แถวที่ผ่านไปตาราง Active แถวที่ไม่ผ่านไป Quarantine พร้อมเหตุผล จากนั้นข้อมูลที่ผ่านแล้วถูกนำไปสร้างแดชบอร์ดด้วย AI หรือส่งออกเป็น CSV ได้ ข้อจำกัดที่เรารู้และบอกอย่างเปิดเผยคือยังไม่มี rollback ยังไม่มีการกระหายอดต้นทางกับปลายทาง และบางส่วนของหน้า Dashboard เป็นข้อมูลจำลอง สรุปคือ SDOQAP ไม่ได้ทำแค่ ETL แต่รวม Data Quality, Quarantine, Monitoring และการนำข้อมูลไปใช้ผ่าน Dashboard เข้าด้วยกัน"

คำถามที่อาจารย์อาจถาม:

1. จุดแข็งของระบบคืออะไร? → Quarantine พร้อมเหตุผลระดับแถว, MERGE ตาม PK (รันซ้ำไม่ซ้ำ), run registry + lock + checksum, Dashboard Builder ที่ LLM ไม่คำนวณตัวเลข และมี validation + fallback

2. จุดอ่อน/สิ่งที่ยังไม่ได้ทำ? → ไม่มี reconciliation ต้นทาง-ปลายทาง, quality score ไม่ขวางไหล, ไม่มี rollback/CDC, แท็บ API/Stream ใน UI เป็นโหมดสาริต, Grafana ไม่มี dashboard, บาง KPI/narrative เป็น mock
3. ถ้าจะให้ระบบใช้งานจริงต้องเพิ่มอะไร? → ตอบจากข้อจำกัดข้างบนเท่านั้น (ยังไม่เสนอการออกแบบใหม่ใน deck นี้)

Presentation Cheat Sheet (1 หน้า — ใช้ซ่อมพุด)

Slide	พูดอะไร (สั้น ๆ)	ตัวเลข/คำที่ต้องจำ
1 ภาพรวม	SDOQAP = ระบบรับ→ตรวจ/แปลง→เก็บ→ใช้ข้อมูล; ของที่ไม่ผ่านแยกเข้า Quarantine ไม่ทิ้ง	Source → Extraction → Transformation → Quality → Loading → Utilization
2 กระบวนการ	21 stage ใน 3 ช่วง จัด 6 กลุ่ม + inline 7 + กระบวนการฝั่ง API ≥ 10 ทั้งหมดอัตโนมัติ	1 + 12 + 8 = 21
3 Extraction	CSV/Excel · REST · PostgreSQL ต่อจริง; ตรวจ auth·ชื่อตาราง·200MB·ว่าง·SHA-256·PK header → HDFS raw → run registry → Spark; Reddit แยกเส้น	ยังไม่เทียบจำนวนต้นทาง · แท็บ API/Stream ใน UI = สาริต
4 Transformation	แปลงชนิด·rename·ลบซ้ำ·ตรวจ null/type/date·มาตรฐานวันที่ (พ.ศ.→ค.ศ.)·IQR/Z-score·แยก Valid/Invalid	ตัวอย่าง: 1, 234→1234 · 22-7-2569→2026-07-22 · users.csv
5 Quality	กฎรายแถวทุกข้อ → Quarantine+เหตุผล; Score = Clean/(Clean+Quarantine)×100 คิดหลังไหล ไม่หยุดไหล	threshold 90 / พื้น 70 / Z-score 3.0
6 Loading	Valid→Delta Active (MERGE by PK) · Invalid→Delta Quarantine (append) · Raw→Archive · Metadata→ES; เขียน Quarantine ก่อน MERGE	ไม่มี rollback · ไม่มี reconciliation · ไม่มี CDC
7 Utilization	Create Dashboard AI: Active→LLM ออกแบบ spec→validate→pandas คำนวณ; LLM ไม่คำนวณเลขและไม่เห็นแถว; Export·Trust-check จริง	Sell-In/Out = mock · ปุ่ม "นำไปใช้" = UI only · Grafana ไม่มี dashboard
8 White Box	Dev=กฎ/ลำดับ · User=ส่งข้อมูล/ตั้งกฎ/อนุมัติ/สร้าง dashboard · Spark=ทำงานจริงทั้งหมด · LLM=เสนอกฎ+ออกแบบ spec	auto-remediation ใช้กฎ conf ≥ 0.80 โดยไม่ต้องอนุมัติ
9 Architecture	React → FastAPI → HDFS → Spark(daemon) → Delta + ES → Dashboard; LLM ช่วย; n8n/Kafka/Grafana บทบาทจำกัด	Postgres = แหล่งเท่านั้น · ES = metadata ไม่ใช่แถวข้อมูล
10 สรุป	วงจรครบ 7 ชั้น; พุดข้อจำกัดเอง; ปิดด้วยประโยค "ไม่ใช่แค่ ETL..."	"Data Quality + Quarantine + Monitoring + Dashboard"

ห้ามพูด (ไม่เป็นจริงตามผลวิเคราะห์):

- X "Quality Score ขววงการโหลด/เป็น gate ก่อนโหลด" → จริง: คัดหลังโหลด
- X "ตรวจจำนวนแถวต้นทางกับปลายทางตรงกัน" → จริง: `total = clean + quarantine` (ไม่ใช่การเทียบต้นทาง)
- X "ดึงจาก API/Kafka ได้จากหน้า UI" → จริง: แท็บ API/Stream เป็นโหมดสาธิต
- X "รองรับ JSON/Parquet/Avro/XML, MySQL/Oracle, incremental CDC, rollback, Delta time-travel"
- X "มี Grafana dashboard / Prometheus ทำงานอยู่ / Slack-LINE ส่งได้ทันที"
- X "Sell-In/Sell-Out, narrative อธิบาย, metadata ของ lineage node เป็นข้อมูลจริง"
- X "ป้อนนำไปใช้ของ Recommendation สั่ง halt/restore จริง"
- X "LLM คำนวณตัวเลขใน Dashboard" / "AI ทุกตัวต้องมีคนอนุมัติก่อนใช้"
- X "รันซ้ำแล้วข้อมูลไม่ซ้ำทุกที" → จริง: Active ไม่ซ้ำ; ES metadata/Gold/Quarantine เส้นเก่าอาจซ้ำ